Cheatsheet AZ-305 — 08 · Cómputo y contenedores

1. El eje PaaS ↔ IaaS: cuándo hace falta una VM

Requisito del enunciado	Respuesta
Acceso administrativo al SO, instalar dependencias a nivel de sistema	**VM / VMSS** (ningún App Service, ni Isolated/ASE, da acceso al SO)
Componente **COM** registrado en el host	**VM / VMSS**
Software legacy con dependencias de SO	**VM / VMSS**
.NET Framework completo + escritura de archivos temporales + Event Log, mínimo mantenimiento	**App Service (Windows)** basta
Aplicación web/API estándar, mínimo mantenimiento y coste	**App Service**
No compartir hardware físico con otros clientes	**Azure Dedicated Hosts** (VMs)
Sobrevivir a un data center	**Availability Zones** (VMSS en 2+ zonas)
Sobrevivir a una región	Dos regiones + Traffic Manager / Front Door

2. App Service: planes y ASE

Tier	Instancias dedicadas	Autoscale / slots	VNet Integration (salida)	Private Endpoint (entrada)	Aislamiento
Free / Shared	✗	✗	✗	✗	—
Basic	✓	✗	✓ (v2)	✓	Multi-tenant
Standard	✓	✓	✓	✓	Multi-tenant
Premium (v2/v3)	✓ más CPU/RAM	✓ más slots	✓	✓	Multi-tenant
Isolated (v2)	✓	✓ hasta 200 inst.	✓	✓	**Solo dentro de un ASE**

- **App Service Environment (ASE):** despliegue completo de App Service **dentro de tu VNet** (IP privada, sin IP pública en ILB ASE, todo el tráfico bajo tus NSGs/UDRs/firewall, hasta 200 instancias). **Sigue siendo PaaS: sin acceso al SO.** Muy caro → solo con aislamiento total, cumplimiento o escala.
- **Tráfico privado App Service → SQL:** VNet Integration (subnet **delegada**, exclusiva) + Private Endpoint de SQL (otra subnet) + Private DNS zone → **mínimo 2 subnets**.
- **Redundancia regional:** un plan vive en una región; multi-región = dos despliegues + Front Door / Traffic Manager.
- **Deployment slots** (Standard+): swap sin downtime. **Health check** para sacar instancias enfermas.
- Configuración repetible y con historial → **Bicep/ARM en Azure Repos** (no ZIP, no imágenes).

3. Azure Functions: planes

Plan	Escalado	VNet Integration	Cold start	Coste	Cuándo
Consumption	Por eventos, a cero	✗ (limitado)	Sí	Mínimo	Código puntual sin red privada
Flex Consumption	Por eventos, a cero	✓	Reducido	Bajo	Serverless con VNet
Premium	Por eventos, instancias pre-calentadas	✓	No	Medio	**Acceder a IP privada** + coste razonable
Dedicated (App Service plan)	Manual/autoscale del plan	✓	No	Fijo	Reusar un plan existente

- **Durable Functions:** Client (arranca) · **Orchestrator** (define flujo, **durable timers**, condicionales) · Activity (trabajo) · Entity (estado).
- **Authorization level:** Anonymous (público) · Function (key) · Admin (master key). Read-only público → **Anonymous + GET only**.
- Trigger de Queue Storage migrado a AKS → **KEDA + HPA**.

4. Contenedores: ACI vs Container Apps vs AKS

	Container Instances	Container Apps	AKS
Unidad	Un contenedor / container group	App con réplicas	Clúster Kubernetes completo
Escalado automático	✗	✓ 0→N (HTTP, CPU, **eventos/KEDA**)	✓ HPA, KEDA, cluster autoscaler
Escalar a cero	Solo borrando	✓	Con KEDA
Ingress / TLS gestionado	✗	✓	Ingress controller que despliegas
Revisiones / canary	✗	✓	Con herramientas (Flux, Istio)
Microservicios / Dapr	✗	✓ integrado	Add-on
Control del clúster	—	Oculto	**Total** (kubectl, Helm, CRDs, redes)
Arranque	Segundos	Segundos	Minutos (nodos)
Facturación	Por segundo CPU/RAM	Por consumo	Nodos (VMs)

	Container Instances	Container Apps	AKS
Caso	Jobs, tareas puntuales, **burst de AKS (virtual nodes)**	APIs, webs, microservicios sin gestionar K8s	Cargas Kubernetes, control fino

- **Service Fabric:** orquestador de microservicios de Microsoft, **híbrido on-prem + Azure**, actualizaciones independientes, auto-reparación, hyperscale.
- **Azure Batch:** cómputo paralelo con scheduler propio. **CycleCloud:** clústeres HPC con **scheduler de terceros** (Slurm, PBS, LSF).
- **Container Registry Premium: geo-replicación** de imágenes para AKS multi-región; CMK; private link.

5. AKS: piezas que caen en el examen

Pieza	Qué hace	Señal
Ingress controller (NGINX, AGIC)	Único punto de entrada L7: paths, hosts, TLS, **mTLS**, puertos no estándar; evita un LB por servicio	"expose multiple APIs", "non-default ports", "mTLS", "min cost"
Service LoadBalancer	Un Azure LB por servicio, L4	Solo si un servicio
ExternalName service	Alias DNS hacia algo externo	Distractor
HPA	Escala pods por CPU/RAM o métricas externas	Con KEDA
KEDA	Escala pods por **eventos** (colas, Service Bus, Event Hubs), 0↔1; crea el HPA	"same scaling as Functions Consumption"
Cluster autoscaler	Escala **nodos**; funciona con kubenet y Azure CNI	"nodes", "capacity"
Virtual nodes / Virtual Kubelet	Pods en ACI para burst; **solo Azure CNI**	Descartado si dicen "kubenet"
Dapr	Building blocks: **state management**, pub/sub, service invocation, secrets	"state management"
Flux	GitOps: sincroniza el clúster con un repo	"deploy from Git"
Istio	Service mesh: mTLS entre pods, canary, tráfico	"traffic between services"
Azure Policy add-on	Gatekeeper/OPA (EnforceOPAConstraint / Rego)	"pod governance"
kubenet vs Azure CNI	kubenet: IPs de pods NAT; CNI: IPs de la VNet (más IPs, integración directa)	"pods reachable from VNet" → CNI

- Front Door → 2 clústeres AKS en 2 regiones con SSL centralizado.
- APIM + AKS mismo VNet, IP privada, mTLS, rate limit (opciones APIM) → Premium Internal.

6. VMs y VMSS: reglas de examen

Tema	Regla
SLA	VM sola con Premium SSD 99.9 % · Availability Set 99.95 % · Availability Zones 99.99 %
Availability Set	Fault / update domains, mismo datacenter; no protege de fallo de datacenter
Availability Zones	Datacenters separados en la región; "data center failure"
VMSS	Escalado **horizontal**; autoscale con métricas, duration, cooldown, **profiles** por horario/fecha; software en aprovisionamiento con **Custom Script / DSC extension** en extensionProfile + script subido
Series	D/Ds general (por defecto y coste), E memoria, F cómputo, L disco local, M memoria masiva, N GPU (NC cómputo, NV visual), H HPC; **"s" = Premium storage**; SR-IOV = Accelerated Networking
Discos	SO: Premium SSD **v1** (v2 no vale para SO); SQL: log **None**, data **ReadOnly**
Dedicated Hosts	Hardware físico no compartido
Confidential VMs	Datos en uso cifrados (TEE)
Escalar sin downtime	Horizontal + VMSS + Availability Set/Zones
JIT VM access	Defender for Cloud abre puertos de gestión temporalmente (no es PIM)
Migración VMware	Azure Migrate appliance (descubrimiento) / AVS solo si quieren seguir en VMware

7. Árbol de decisión

```
¿Qué hosting?
├─ Necesita SO / COM / dependencias de host ..... VM / VMSS (zonas para datacenter)
├─ Web/API estándar, min mantenimiento ........... App Service (Standard/Premium)
├─ Igual pero sin IP pública / aislamiento total .. ASE (sigue sin SO)
├─ Código por eventos ............................ Functions (Premium si IP privada)
├─ Contenedor puntual / job ...................... Container Instances
├─ Microservicios sin gestionar K8s .............. Container Apps
├─ Kubernetes con control ........................ AKS
├─ Microservicios híbridos on-prem + Azure ....... Service Fabric
├─ HPC con scheduler de terceros ................. CycleCloud
└─ Cómputo paralelo gestionado ................... Batch
```

8. Señales → respuesta

Enunciado	Respuesta
"administrators access to the OS"	VMs / VMSS (nunca App Service)
"custom COM component" + "data center unavailable" + "min cost"	VMSS + LB en 2 availability zones
"Isolated plan" + "redundancy if region fails"	No cumple: una región, sin SO
".NET web service, temp files, event log, min cost"	App Service web app
"Event Grid + C# + private IP + min cost"	Functions Premium
"state management" (AKS)	Dapr
"same scaling as Consumption" (AKS, kubenet)	KEDA + HPA
"expose APIs on non-default ports + mTLS + min cost"	Ingress controller
"image replicated to AKS in multiple regions"	Container Registry Premium (geo-replication)
"HPC + third-party scheduler"	CycleCloud
"on-prem + Azure microservices, self-healing"	Service Fabric
"define config of 50 App Services + version history"	Bicep in Azure Repos
"15,000 IOPS + SR-IOV + min cost"	Serie DS

9. Errores a evitar

1. ASE/Isolated como respuesta a "acceso al SO".
2. Dos regiones cuando piden sobrevivir a un data center y minimizar coste → zonas.
3. Virtual nodes con kubenet.
4. HPA solo (CPU) para replicar Functions Consumption → KEDA + HPA.
5. Container Instances para una API con autoscaling → Container Apps.
6. Escalado vertical en VMSS.